

哨兵 EdgeSentinel · D2 成果报告

先验校正 · CUDA 决策头 · 工业决策服务 | NVIDIA DGX Spark GB10 | 2026-09-22

0 D2 完成概览

#	任务	结果	对应评分点
1	先验校正：修掉校准的分布漂移	ECE 0.1722 → 0.0500 (改善 71%)	25% 技术深度
2	决策头移植到 GB10 GPU (torch CUDA)	单条 0.0339 ms , 比 CPU 方案快 2.9×	15% 平台适配 (Spark 利用率)
3	工业决策服务 + 实时看板	公网单端口可演示：三条类型化决策 + 路由 + 复核队列	20% 完整性 / 10% 演示

本轮还修掉了两个会直接影响答辩的架构缺陷：① 三个决策头独立训练导致输出自相矛盾 (已加一致性约束) ；② 路由判据用错 (应为置信度 $\max(p,1-p)$, 不是 p 本身) 。详见第 3 节。

1 先验校正：让校准不再依赖部署分布

此前主动暴露的问题：同一个决策头，在自然分布下 ECE 仅 0.0061，换到均衡分布就劣化到 0.1722。根因是模型输出的后验隐含了训练分布先验 ($\pi_{\text{train}} = 0.0312$) 。标准解法是先验校正： $z' = z + \log(\pi_{\text{deploy}}/(1-\pi_{\text{deploy}})) - \log(\pi_{\text{train}}/(1-\pi_{\text{train}}))$ 。

场景	ECE	Brier	平均预测概率	实际故障率
A 自然分布 (与训练一致)	0.0061	0.0155	—	0.0350
B 均衡分布 · 未校正	0.1722	0.1157	0.3278	0.5000
C 均衡分布 · 先验校正后	0.0500	0.0568	0.4883	0.5000
E 先验未知 · EM 盲估	0.0645	0.0570	—	0.5000

B 行暴露了失配的本质：真实故障率 50%，模型平均只预测 0.3278，系统性低估。校正后平均预测回到 0.4883，ECE 下降 71%，Brier 从 0.1157 降到 0.0568。

1.1 敏感性：先验猜错了会怎样

校正时假设的先验	ECE	Brier	Accuracy
0.05	0.1452	0.0936	0.8857
0.10	0.1025	0.0709	0.9214
0.20	0.0810	0.0591	0.9429
0.30	0.0585	0.0562	0.9286
0.40	0.0617	0.0558	0.9286
0.50 (真实值)	0.0500	0.0568	0.9357
0.60	0.0592	0.0591	0.9286
0.70	0.0703	0.0636	0.9143
不校正 (基准)	0.1722	0.1157	约 0.83

关键结论：即使先验猜错，也比不做校正好。最差的 0.05 档 ECE 0.1452，仍优于不校正的 0.1722，方法足够稳健。而真实先验未知时，EM 算法只用无标签部署样本盲估到 0.2562（真实 0.5），把 ECE 压到 0.0645——这意味着线上可以在没有标注的情况下自校准。

2 CUDA 决策头：决策层上 GB10

节点 torch 2.14.0+cu130 可用后，把三个决策头移植为 PyTorch 模型，直接跑在 GB10 上。三个头 800 epoch 训练仅需 2.8 秒。

任务	torch 版 Accuracy	sklearn 版基线	差异
Noul (是否故障)	0.9745	0.9775	-0.0030
Choice (故障类型)	0.9650	0.9805	-0.0155
Score (维护优先级)	0.9850	0.9890	-0.0040

torch 版与 sklearn 版在 Noul 上仅差 0.3 个百分点，Score 差 0.4 个点；Choice 差 1.6 个点（少数类样本不足所致，与实现无关）。精度迁移基本无损。

2.1 延迟与吞吐

batch	GPU ms/batch	GPU 条/秒	CPU ms/batch	CPU 条/秒	加速比
1	0.0344	29,075	0.2137	4,679	6.2×
16	0.0348	459,992	0.8724	18,339	25.1×
64	0.0514	1,245,751	0.9599	66,674	18.7×
256	0.0384	6,666,445	0.9004	284,317	23.4×
1,024	0.0369	27,757,095	1.4161	723,101	38.4×
4,096	0.0357	114,828,241	1.8493	2,214,914	51.8×
16,384	0.0544	301,090,866	2.2770	7,195,275	41.8×

单条：GPU 0.0339 ms vs torch-CPU 0.2520 ms；对照现有 sklearn CPU 决策头 0.097 ms，GPU 版快 2.9 倍。批量峰值 301,090,866 条/秒（batch 16,384）。GB10 是统一内存架构，没有独立显存搬运，因此连 batch=1 的 kernel launch 开销都被压到 34 μ s 级别。

统一内存实测：单次前向可容纳 batch $\approx$ 2,097,152（约 200 万条设备状态），稳定吞吐约 3,800 万条/秒。这是 Spark 相对独立显存设备的差异化能力——一次把整座工厂的历史数据全量重算是可行的。

3 工业决策服务：System One 上线

项目	实现
部署位置	gx10-902c 节点 :9000，公网 http://61.172.235.130:9014
推理设备	GB10 GPU (cuda)，CPU 为回落路径
端点	GET /healthz GET / (看板) POST /v1/decide POST /v1/decide/batch
输入	设备状态：气温、过程温度、转速、扭矩、刀具磨损（摄氏度 + 常用单位）
输出	三条类型化决策（Noul / Choice / Score）+ 校准概率 + 路由动作 + 原因码
可选参数	prior_deploy（先验校正）、th_auto、th_review（路由阈值）

项目	实现
管理	cd ~/jev-service && ./sentinelctl.sh start stop restart status logs

3.1 修掉的两个架构缺陷

缺陷	表现	修正
决策之间自相矛盾	Noul 判「99.3% 是故障」，Choice 却判「64% 无故障」——三个头独立训练，输出不自洽	一致性约束： $P(\text{No Failure})=1-p$ ， $P(\text{故障类型 } k)=p \times \text{该类占比}$ 。修正后 No Failure 概率正好等于 $1-p$ ，两个决策永远自治
路由判据用错	原写法用 $p \geq \text{阈值}$ 判自动派单，导致 auto_dispatch 1000 条里只触发 10 次	改为置信度 $= \max(p, 1-p) \geq \text{阈值}$ 才自动化——这才对应「阈值—自动化率—准确率」权衡表的语义

第二个缺陷尤其重要：它让我们自己那份权衡表和服务行为对不上。修正后批量实测自动化率 71.1%，三个分支（auto_close / auto_dispatch / human_review）均正常触发。

3.2 服务实测（公网调用，1000 条批量）

指标	实测值
模型推理耗时（1000 条）	0.84 ~ 0.89 ms
每条延迟	0.00085 ms
吞吐	约 1,180,000 条/秒
自动化率	71.1%（自动 711 / 复核 289）
路由分布	auto_close 701 · human_review 289 · auto_dispatch 10
单条端到端（含 HTTP）	1.2 ~ 4.1 ms（首次含 CUDA warmup）

3.3 典型样本的决策行为

场景	Noul 概率	置信度	故障类型	严重度	路由
明显故障（磨损 243min、温升 13.5K、扭矩 62.3）	0.9928	0.9928	TWF	立即	human_review
健康（磨损 5min、温升 8.1K）	0.0000	1.0000	No Failure	计划	auto_close
边界（磨损 208min、温升 14.2K）	0.7876	0.7876	PWF	尽快	human_review

第一条虽然 Noul 概率高达 0.9928，仍被判为 human_review——因为严重度是「立即」，策略层覆盖模型判断，强制转人工。这正是整套架构的安全边界：模型只负责判断，行动由确定性代码决定。

3.4 实时看板

服务根目录直接托管看板页面（同源、零外部依赖，节点无外网也能跑）：报警流每 1.2 秒推一条（可提速至 $\times 3 / \times 8$ ），实时展示三个决策的概率条、路由动作、人工复核队列与自动化率统计。传感器数据由页面内仿真器生成，约 12% 注入真实故障特征（按 AI4I 的故障规律）。

4 下一步（按性价比排序）

#	事项	价值	阻塞条件
1	接入 StepFun API（合成数据 / 难例终审 / 报告生成）	15% 平台适配里的白送分项	需要 StepFun API key

#	事项	价值	阻塞条件
2	Nsight Systems / Compute 剖析，出 kernel 级报告	NVIDIA 技术栈硬证据	无，可直接做
3	演示视频：本地决策层 vs 27B 生成式 的实时对比	10% 演示（真实权重远超 10%）	需你出镜/录屏
4	补 CWRU / XJTU-SY 数据集，解决少数类（TWF/RNF）	提升 Choice 宏平均 F1 0.505	需下载数据集
5	征文（每日 15 分钟，最后合成「十日谈」）	5% 征文	无

复现：cd ~/jev-service && python3 prior_calib.py / gpu_head.py | 服务：./sentinelctl.sh restart | 全部数据源自 gx10-902c 真机实测